

1-编程题：多数投票分类器

本次实验比较简单，就是少数服从多数原则，本次作业中不需要看特征，只看类别标签；比如心脏病数据集，不看什么血糖，血压啥的特征，只看心脏病的训练集中，如果得病的占大多数，那这个Majority_vote就输出得病。训练好了后，对于测试集上的任何样本，Majority_vote都输出得病的判断。

注意：这里其实并不算训练，它没有什么参数需要学习，更像是统计数据

majority_vote.py多数投票分类器

在训练集上进行学习【这里的学习其实是统计】，且都是二分类问题

- 对于heart数据集来说：

统计heart_train.tsv中最后一列heart_disease中的0和1谁更多，更多的那个作为多数投票分类器学习到的类别。这里是0的数量>1的数量，所以多数投票分类器的类别是0

然后在heart_train.tsv和heart_test.tsv上进行分类，即把他们都预测为0类别

计算指标：error = 正确预测的个数/样本的总数

heart_metrics.txt

```
error(train): 0.49
error(test): 0.4020618556701031
```

同理对于education数据集：

education_metric.txt

```
error(train): 0.325000
error(test): 0.310000
```

2-概率统计

$$P = \left(\frac{3}{4}\right)^4 \times \frac{1}{4} \times 5 = \frac{405}{1024}$$

$$2^{\frac{4}{5}}$$

设 $p = p(x = 1)$, $f(p)$ 是观测到出现 S 的概率, 则

$$\begin{aligned} f(p) &= p^4(1-p) \\ f'(p) &= 0 \end{aligned}$$

$$\text{得 } p = \frac{4}{5}$$

3 A

容斥原理

4 A

贝叶斯公式有:

$$P(A(BC)) = P(A \mid BC)P(BC) = P(A \mid BC)P(B \mid C)P(C)$$

5C

$$P(W) = P(WR) + P(W\bar{R}) = P(R)P(W \mid R) + (1 - P(R))P(W \mid \bar{R}) = 0.8 \times 0.4 + 0.2 \times 0.6 = 0.44$$

6B

$$P(x = 1 \mid y = 1) = \frac{P(x = 1, y = 1)}{P(y = 1)} = \frac{3}{7}$$

7D

全概率公式

$$P(Y = 0) = \sum_x P(Y = 0, X = x) = 0.1 + 0.2 = 0.3$$

8C

$$E(X) = \int_{-\infty}^{+\infty} x f(x) dx$$

$$D(X) = \int_{-\infty}^{+\infty} (x - E(x))^2 f(x) dx$$

所以

$$E(6X) = \int_{-\infty}^{+\infty} 6x f(x) dx = 6E(X)$$

$$D(2X) = \int_{-\infty}^{+\infty} (2x - 2E(x))^2 f(x) dx = 4D(X)$$

$$D(2X - 3) = D(2X)$$

9C

10.D

11.AB

$$P(A = a_1, B = b_1) \neq P(A = a_1) P(B = b_1)$$

$$P(C = c_1, B = b_1) \neq P(C = c_1) P(B = b_1)$$

12 N/A

由于

$$P(B = b_1 | A = a_2, C = c_4) = \frac{P(B = b_1, A = a_2, C = c_4)}{P(A = a_2, C = c_4)}$$

而 $P(A = a_2, C = c_4)$ 未知, 所以概率无法求解

13 0

注意到

$$P(B = b_2, A = a_3, C = c_3) = P(A = a_3 | B = b_2, C = c_3) P(B = b_2, C = c_3) = 0$$

所以

$$P(B = b_2 | A = a_3, C = c_3) = \frac{P(B = b_2, A = a_3, C = c_3)}{P(A = a_3, C = c_3)} = 0$$

14 A

因为当 $C = c_1$ 时, B 所有的取值 (b_1, b_2, b_3) 的概率被相加, 所以左边为 1, 同理, 右边也为 1

$$LHS = 1 = RHS$$

15C

$$\begin{aligned} P(X = x, Y = y) &= P(Y = y | X = x)P(X = x) = \frac{1}{n \times 2^x} \\ P(Y = y) &= \sum_{x=1}^{\infty} P(X = x, Y = y) = \frac{1}{n} \\ E(Y) &= \sum_{y=1}^{\infty} yP(Y = y) = \sum_{y=1}^n \frac{y}{n} \end{aligned}$$

16 A

$$H(x) = \sum -p_k \log_2(p_k)$$

17 A Multivariate Gaussian: B Exponential: C Binomial: D Uniform: E Bernoulli:

3-微积分

1

$$\begin{aligned} y' &= \frac{-\frac{12}{x^4} - 2x}{\frac{4}{x^3} - x^2} \\ y'|_{x=1} &= -\frac{14}{3} \end{aligned}$$

2C

$$\frac{\partial y}{\partial x} = \sin z (3x^2 e^{-x})' = 3x \sin z e^{-x} (2 - x)$$

3 B

显然 $f(x)$ 有二阶导数, 由泰勒公式

$$\begin{aligned} x_0 &= -\frac{1}{6} \\ f(x) &= f(x_0) + \frac{1}{2} f''(c)(x - x_0)^2 \leq f(x_0) \end{aligned}$$

这说明在 x_0 是其邻域内的最大值

4 A

θ 是一个向量, $g(\theta)$ 是一个值

4-矩阵论

1D

2 B

3AB

4D

5 A

$$|A| = -58 \neq 0$$

6 B

$$z = \sum_{i=1}^3 x_i y_i$$

7C

$$Xy = 2y$$

8D

$$[3, 117, 9]^T = 3[1, 39, 3]^T$$

它们都是

$$Av = 4v$$

的解

5-Geometry

1 B

$$\begin{aligned} \mathbf{w}^T \mathbf{x}' + b &= 0 \\ \mathbf{w}^T \mathbf{x}'' + b &= 0 \\ \mathbf{w}^T (\mathbf{x}' - \mathbf{x}'') &= 0 \end{aligned}$$

2 A

点到直线的距离公式:

$$d = \frac{|b|}{\sqrt{w_1^2 + w_2^2}}$$

3 A

6-CS基础

1C

根据换底公式 $\log_a b = \frac{\log_c b}{\log_c a}$, 有 $\ln n = \frac{\log n}{\log e}$, $\log_3 n = \frac{\log n}{\log 3}$, 则
 $O(\ln n) = \log n = O(\log_3 n)$

2 A

指数增长具有爆炸性特征, 要远远快于多项式增长

3 B

4

dfs: 1, 2, 3, 5, 6, 4, 7, 8, 9, 10

bfs: 1, 2, 7, 3, 4, 8, 5, 6, 9, 10

5

```
def find_val(node, key):  
    if node is None:  
        return None  
    if node.val == key: # (1) Check if the current node's value  
matches the key  
        return node  
    else:  
        result = find_val(node.leftNode, key) # (2) Recursively  
search the left subtree  
        if result is None:  
            result = find_val(node.rightNode, key) # (3) Recursively  
search the right subtree  
        return result # (4) Return the result of the search, either  
the found node or None
```

6C

```
def fib_1(n):  
    if n == 1:  
        return 1  
    if n == 0:  
        return 2  
    return fib_1(n-1) + fib_1(n-2)  
  
print(fib_1(32))
```

7C

fib_1(n)是以时间换空间, fib_2(n)是以空间换时间

$$T(n) = 2T(n - 1)$$

故

$$T(n) = O(2^n)$$

8 A

由于每一个值只被计算一次, 时间上相当于递推

$$T(n) = O(n)$$

[三种时间复杂度算法求解斐波那契数列 - 知乎 \(zhihu.com\)](https://www.zhihu.com/question/26827113/answer/111111111)